INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

Fellowship in Cybersecurity & Digital Forensics

LAB REPORT

CIP-B101: Basic Computer Skills for Digital Forensics

Lab 1B: PC System Fundamentals, Disk Sectors and Partition Analysis

Safe, read-only profiling of a computer system: evidence layers, processes, sector arithmetic, partition tables, boot process and file systems

Student Name	Samina Parveen
Registration No.	C11/26/FCDF/17139
Programme	ICDFA Fellowship in Cybersecurity & Digital Forensics
Course Code	CIP-B101 – Basic Computer Skills for Digital Forensics
Lab	Lab 1B – PC System Fundamentals, Disk Sectors and Partition Analysis
Date	7 November 2026
Platform Used	Kali Linux (VMware Virtual Machine)
Working Folder	~/cip-b101-laba1_Samina_Parveen

Submitted as: CIP-B101_Lab1B_Samina_Parveen.pdf

Table of Contents

1. Lab Overview..... 3
1.1 Tools Used..... 3
1.2 Safety and Evidence Handling..... 3
2. Part A – System Identification and Evidence Layer Mapping..... 4
2.1 A1 – Basic system specification..... 4
2.2 A2 – Storage information..... 5
2.3 A3 – Evidence layer mapping..... 6
3. Part B – Program, Process and Thread Investigation..... 7
3.1 B1 – Observe running processes..... 7
3.2 B2 – Program, process and thread..... 8
4. Part C – Disk Sector and Disk Size Calculation..... 8
4.1 C1 – Reproduce the slide calculation..... 8
4.2 C2 – Calculate my own partition and disk size..... 8
5. Part D – Safe Partition Table Inspection..... 10
6. Part E – MBR, GPT and Boot Process Explanation..... 11
6.1 E1 – PC boot process..... 11
6.2 E2 – Compare MBR and GPT..... 11
7. Part F – File System Identification and Interpretation..... 12
8. Part G – Final Practical Assignment..... 12
9. Conclusion..... 14
10. Screenshot Checklist..... 14

1. Lab Overview

Scenario: I am a junior digital forensic trainee asked to prepare an initial system profile of a computer that may later be used in an investigation. The goal of this lab is not to change anything on the machine. It is to observe, record, calculate, verify and document findings in a repeatable way.

In this report I identify the hardware and operating system, map where digital evidence can exist, observe running processes, calculate disk and partition sizes from raw sector counts, inspect the partition table with read-only commands, explain the PC boot process and MBR/GPT, and identify the file systems in use.

1.1 Tools Used

Tool / Command	Purpose	Used in
`uname -a, hostnamectl`	Kernel, OS, architecture, virtualization and firmware details	Part A1
`sudo fdisk -l`	Disk model, sector size, partition table type, start/end/sector count	Parts A2, C, D
`lsblk -f, df -Th`	Block devices, file system types, UUIDs, mount points, usage	Parts A2, F
`ps -eo pid,ppid,cmd,%mem,%cpu`	Running processes with PID, parent PID and resource use	Part B
`sudo parted -l`	Independent check of disk size, table type and partition flags	Part D
`Python 3 / bc (calculator)`	Sector-to-GB arithmetic	Part C

1.2 Safety and Evidence Handling

Every command used in this lab is read-only. I did not format, delete, resize, encrypt or create any partition, and I did not run `mkfs`, `dd`, or any write command in `fdisk` or `parted`. The commands only list information, so the virtual disk is unchanged by this work. The lab was done inside my own Kali Linux training virtual machine running on VMware.

The working folder carries my name so that every screenshot identifies the student: `~/cip-b101-laba1_Samina_Parveen` (visible in the prompt of every screenshot).

2. Part A – System Identification and Evidence Layer Mapping

2.1 A1 – Basic system specification

The lab manual gives Windows tools (`msinfo32`, `systeminfo`) for this step. My system is a Kali Linux VM, so I used the Linux equivalents `uname -a` and `hostnamectl`, which report the same kind of information.

```
$ uname -a && hostnamectl
```

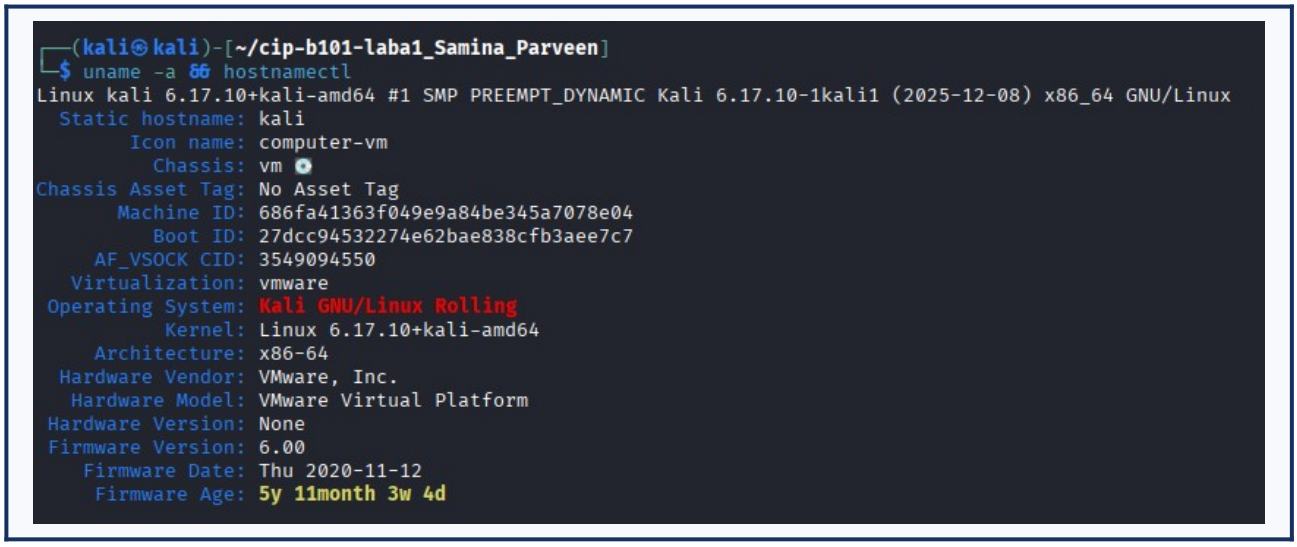


Figure 1: *System information from uname -a and hostnamectl.*

Important observations

Item	Value observed
Operating system	Kali GNU/Linux Rolling
Kernel	Linux 6.17.10+kali-amd64 (SMP PREEMPT_DYNAMIC, built 2025-12-08)
Architecture	x86-64
Hostname / chassis	kali / vm
Virtualization	vmware – Hardware vendor VMware, Inc., model VMware Virtual Platform
Firmware	Version 6.00, dated Thu 2020-11-12 (virtual BIOS provided by VMware)

The machine is a virtual machine, not physical hardware. That matters in forensics because the "disk" is really a virtual disk file on the host, and the hardware identifiers (vendor, model, firmware) belong to the hypervisor rather than to a real manufacturer.

2.2 A2 – Storage information

```
$ sudo fdisk -l
$ lsblk -f
$ df -Th
```

```
┌──(kali㉿kali)-[~/cip-b101-laba1_Samina_Parveen]
└─$ sudo fdisk -l
[sudo] password for kali:
Disk /dev/sda: 80.09 GiB, 86000000000 bytes, 167968750 sectors
Disk model: VMware Virtual S
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0×febdb644

Device     Boot Start       End   Sectors  Size Id Type
/dev/sda1  *     2048 167968749 167966702 80.1G 83 Linux
```

Figure 2: *Disk model, size, sector size and partition entry from sudo fdisk -l.*

```
┌──(kali㉿kali)-[~/cip-b101-laba1_Samina_Parveen]
└─$ lsblk -f
df -Th
NAME   FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
sda
└─sda1 ext4   1.0   root  4111f04f-67ad-4d5d-a01b-b6cc151feb35   27.9G    59% /
Filesystem     Type      Size  Used Avail Use% Mounted on
udev           devtmpfs  865M     0  865M   0% /dev
tmpfs          tmpfs     194M  1.3M  193M   1% /run
/dev/sda1      ext4       79G   47G   28G  63% /
tmpfs          tmpfs     969M  4.0K  969M   1% /dev/shm
none           tmpfs     1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs          tmpfs     969M  372K  968M   1% /tmp
none           tmpfs     1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
tmpfs          tmpfs     194M  120K  194M   1% /run/user/1000
```

Figure 3: *File system type, UUID, mount point and usage from lsblk -f and df -Th.*

Important observations

- **Disk model:** VMware Virtual S, device `/dev/sda`.
- **Disk size:** 80.09 GiB = 86,000,000,000 bytes = 167,968,750 sectors.
- **Sector size:** 512 bytes logical and 512 bytes physical.
- **Partition table:** Disklabel type **dos** (MBR) with one primary Linux partition, `/dev/sda1`, flagged bootable (*), partition type Id 83.
- **File system:** sda1 is **ext4** (version 1.0, label "root") and is mounted at `/`. Free space is 28G (63% used according to df).

2.3 A3 – Evidence layer mapping

User activity is recorded through the applications used, the operating system, and what is left on disk and in memory (User activities = Applications + Operating System + Disk/Memory). The table below maps five layers to evidence that can be found on this Linux system.

Layer / Source	Example evidence	Where it may be found	Why it matters
User layer	Typed commands, opened files, searched terms	Shell history (~/.zsh_history, ~/.bash_history), recent files, browser history	Links a specific user action to system activity
Application layer	Browser cache, application config, chat logs, document metadata	~/.config, ~/.cache, /var/lib, application databases	Shows what software created or used the evidence
OS layer	System logs, login records, installed packages, user accounts	/var/log, /etc/passwd, journald, package database	Shows system events and account activity over time
Disk layer	Deleted files, file fragments, partition table, slack space	File system on /dev/sda1, unallocated space, sector 0 (MBR)	Supports recovery and timeline analysis
Memory / process layer	Running malware, PIDs, network connections, loaded keys	RAM capture, process list (ps), /proc	Shows live system behaviour that may not exist on disk

Why evidence may exist in more than one layer: a single user action normally leaves traces in several places at once. For example, downloading a file leaves a history entry in the application layer (browser), connection and service records in the OS layer (logs), the file itself in the disk layer, and the running process and its connections in the memory layer. If one trace is deleted or altered, the others can still confirm the event, which is why an examiner correlates all layers instead of relying on one.

3. Part B – Program, Process and Thread Investigation

3.1 B1 – Observe running processes

I listed the 20 most CPU-active processes with PID, parent PID (PPID), command, memory and CPU use:

```
$ ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -n 20
```

```
┌──(kali㉿kali)-[~/cip-b101-laba1_Samina_Parveen]
└─$ ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -n 20
    PID    PPID CMD                         %MEM %CPU
  20285    2105 ps -eo pid,ppid,cmd,%mem,%c  0.2  100
   1040    1028 /usr/lib/xorg/Xorg :0 -seat  7.6  1.4
   2105    2076 /usr/bin/zsh                 0.4  0.7
   1623    1517 xfwm4                        5.8  0.4
   1685    1659 /usr/lib/x86_64-linux-gnu/x  3.0  0.3
   1764       1 /usr/bin/vmtoolsd -n vmusr   2.1  0.2
   1687    1659 /usr/lib/x86_64-linux-gnu/x  1.4  0.2
   2076       1 /usr/bin/qterminal           3.3  0.2
    463       1 /usr/bin/vmtoolsd            0.5  0.2
   1031       1 /usr/bin/containerd          2.4  0.1
     42       2 [kworker/u514:1-kvfree_rcu_  0.0  0.1
   1675    1517 xfdesktop                    4.2  0.1
      1       0 /sbin/init splash            0.7  0.0
    261       2 [kworker/1:2-events]         0.0  0.0
   1055       1 /usr/sbin/dockerd -H fd://   3.8  0.0
   1466       2 [kworker/3:3-mm_percpu_wq]   0.0  0.0
    591       1 /opt/Elastic/Agent/elastic-  2.7  0.0
    412       2 [kworker/0:2-events]         0.0  0.0
     31       2 [kworker/2:0-events]         0.0  0.0
```

Figure 4: *Running processes with PID and PPID (top 20 by CPU use).*

Process observation table

Process name	PID	PPID	Likely purpose
`/usr/lib/xorg/Xorg :0`	1040	1028	X display server that draws the graphical desktop (uses 7.6% memory in the capture).
`xfwm4`	1623	1517	Window manager of the Xfce desktop environment.
`/usr/bin/qterminal`	2076	1	Terminal emulator in which these commands were typed; its child /usr/bin/zsh (PID 2105) started the ps command itself (PID 20285).
`/usr/bin/vmtoolsd`	1764	1	VMware Tools service that links the guest to the host (clipboard, display, time sync).
`/usr/bin/dockerd and containerd`	1055 / 1031	1	Docker container engine and its runtime; may hold containers with their own evidence.
`/opt/Elastic/Agent/elastic-…`	591	1	Elastic Agent: a monitoring/telemetry agent that collects logs and security events.

The parent-child relationship is itself evidence. In the capture, `ps` (20285) was started by `zsh` (2105), which was started by `qterminal` (2076). Following PPID values lets an investigator rebuild how a suspicious process was launched.

3.2 B2 – Program, process and thread

- **Program:** a passive file stored on disk that contains instructions, for example `/usr/bin/qterminal`. It uses disk space but does nothing until it is started.
- **Process:** a program that is running. The operating system gives it a unique PID, its own memory space and resources. PID 2076 is the running instance of the qterminal program.
- **Thread:** the smallest unit of execution inside a process. One process can run several threads at once, and they share the memory of that process.

Forensic relevance: during live forensic triage and malware analysis, the process list shows what is actually executing right now, including malware that exists only in memory and was never saved to disk. From a PID an examiner can see the parent process, the executable path, open files and network connections. Unexpected names, odd parent-child chains (for example a document viewer spawning a shell) or processes running from temporary folders are typical indicators of compromise. Because a process list disappears at shutdown, it is volatile evidence that should be captured early.

4. Part C – Disk Sector and Disk Size Calculation

> **Key formula**
> Disk or partition size = number of sectors × bytes per sector. To convert bytes to GB divide by 1024 × 1024 × 1024.

4.1 C1 – Reproduce the slide calculation

Values from the slide: sectors per track = 400, heads = 12, cylinders = 17000, bytes per sector = 512.

Item	Formula / value	Answer
Total sectors	400 × 12 × 17000	81,600,000 sectors
Total bytes	81,600,000 × 512	41,779,200,000 bytes
Kilobytes	41,779,200,000 / 1024	40,800,000 KB
Megabytes	40,800,000 / 1024	39,843.75 MB
Gigabytes	39,843.75 / 1024	38.91 GB

4.2 C2 – Calculate my own partition and disk size

The sector values below come directly from the `sudo fdisk -l` output in Figure 2. The sector size is 512 bytes.

Item	Start sector	End sector	Sectors	Size (fdisk)	File system
Partition 1 – /dev/sda1	2048	167,968,749	167,966,702	80.1G	ext4 (type 83)
Whole disk – /dev/sda	0	167,968,749	167,968,750	80.09 GiB	–

Calculation 1 – Partition 1 (/dev/sda1)

```
Sector count   = 167,966,702        (also = 167,968,749 - 2,048 + 1)
Bytes          = 167,966,702 x 512 = 85,998,951,424 bytes
Kilobytes      = 85,998,951,424 / 1024 = 83,983,351 KB
Megabytes      = 83,983,351 / 1024     = 82,014.99 MB
Gigabytes      = 82,014.99 / 1024      = 80.09 GB (GiB)
Shortcut: 167,966,702 x 512 / 1024 / 1024 / 1024 = 80.09 GB
```

Calculation 2 – Whole disk (/dev/sda)

```
Sector count   = 167,968,750
Bytes          = 167,968,750 x 512 = 86,000,000,000 bytes
Gigabytes      = 86,000,000,000 / 1024 / 1024 / 1024 = 80.09 GB (GiB)
(in decimal units this is 86.0 GB)
```

Calculation 3 – Space before partition 1 (cross-check)

```
Gap = 167,968,750 - 167,966,702 = 2,048 sectors
Gap = 2,048 x 512 = 1,048,576 bytes = 1 MiB
Check: 2,048 (gap) + 167,966,702 (sda1) = 167,968,750 = whole disk  [matches]
```

The 2,048 sectors in front of partition 1 hold the MBR in sector 0 and unused space that the partitioning tool leaves for 1 MiB alignment. This is why `parted` reports the partition start as 1049 kB (2,048 × 512 = 1,048,576 bytes). It also shows that the partition accounts for every sector of the disk except this small gap.

> **Note on partition 2**
>
> The manual asks for at least two partitions. My virtual disk contains only **one** partition (/dev/sda1), so a second partition does not exist and I did not create one, because this lab forbids changing the disk. For comparison I calculated the whole disk and the 1 MiB gap, and I describe the other file systems visible on the system in Part F.

5. Part D – Safe Partition Table Inspection

I used the Linux read-only method (D2). The commands only list information; I did not run `mkfs`, `dd`, mount read-write, or any write command in `fdisk` or `parted`.

```
$ sudo fdisk -l
$ lsblk -f
$ df -Th
$ sudo parted -l
```

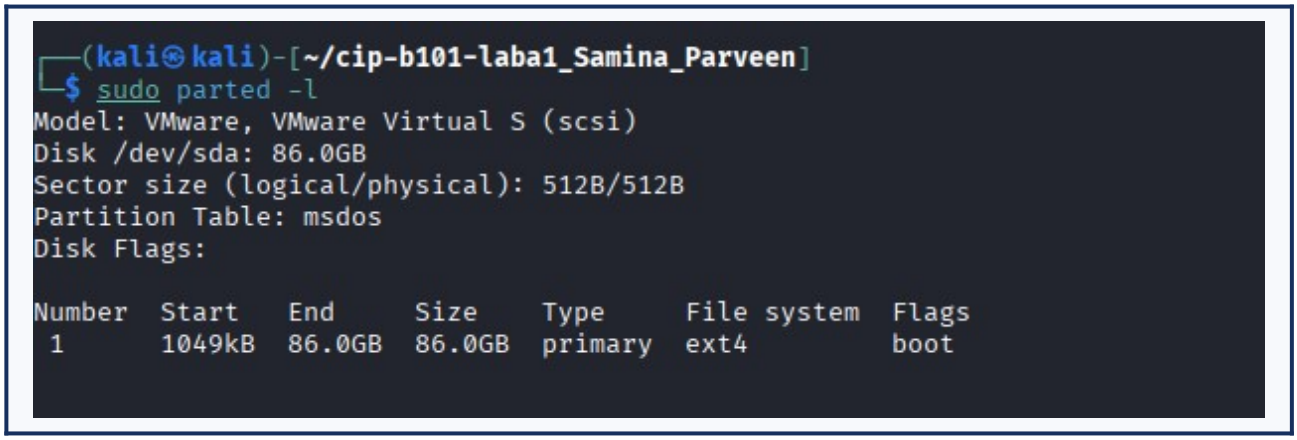


Figure 5: *Independent partition table check using sudo parted -l.*

Item	fdisk (Figure 2)	parted (Figure 5)
Disk model	VMware Virtual S	VMware, VMware Virtual S (scsi)
Disk size	80.09 GiB, 86,000,000,000 bytes	86.0GB
Sector size	512 / 512 bytes	512B / 512B
Partition table	Disklabel type: dos	msdos
Partition 1	/dev/sda1, start 2048, end 167968749, Id 83 Linux	Number 1, start 1049kB, end 86.0GB, primary, ext4
Flags	Boot flag set (*)	boot

Both tools agree: the disk uses an MBR (msdos) partition table and holds one primary, bootable ext4 partition. The only difference is the units. fdisk shows sectors and binary units (GiB) while parted shows decimal units (kB, GB), so 86.0 GB and 80.09 GiB describe the same disk.

6. Part E – MBR, GPT and Boot Process Explanation

6.1 E1 – PC boot process

1. Power-on and POST	Power reaches the board; the firmware runs the Power-On Self-Test to initialize and check CPU, memory and devices.
2. BIOS / UEFI firmware	Firmware settings (stored in CMOS/NVRAM) and the boot device order are applied to find a bootable device.
3. MBR / GPT or EFI boot record	Legacy BIOS reads sector 0 (the MBR) of the disk; UEFI reads the GPT and starts a boot entry from the EFI System Partition.
4. Boot loader / manager	The boot loader (GRUB on most Linux systems) runs, can offer an OS choice and loads the kernel and initial RAM disk.
5. Kernel loading	The kernel starts, initializes hardware, and mounts the root file system (here ext4 on /dev/sda1).
6. Operating system control	The kernel starts the init process (PID 1, /sbin/init in Figure 4); services and the login session start and logs begin to accumulate.

Figure 6: *PC boot sequence from power-on to operating system control.*

Stage	Forensic relevance
Power-on and POST	Hardware failure or tampering may affect access to evidence.
BIOS/UEFI firmware	Boot mode explains the partition style and boot artifacts.
MBR/GPT or EFI boot	Boot records may hold evidence of boot configuration.
Boot loader/manager	May show multi-boot or hidden operating system behaviour.
Kernel and OS control	Logs and user activity begin to accumulate.

Which boot mode does my system use? The evidence from my own system is the partition table: fdisk reports "Disklabel type: dos" (Figure 2) and parted reports "Partition Table: msdos" (Figure 5). UEFI systems normally use GPT with an EFI System Partition, and my disk has neither a GPT nor an EFI partition (it has one single ext4 partition with the boot flag). This indicates that the VM boots in BIOS / Legacy mode with an MBR, which is consistent with the VMware virtual firmware shown in Figure 1.

6.2 E2 – Compare MBR and GPT

Feature	MBR	GPT
Age / design	Older partitioning system from the 1980s.	Modern partitioning system used with UEFI systems.
Size limitation	Commonly limited to about 2 TB.	Supports very large disks (about 9.4 ZB).
Partition information	Partition table is stored in the first sector together with the boot code (4 primary entries).	Uses primary and backup GPT headers and partition entries (a protective MBR sits in sector 0).
Forensic issue	Critical information is in sector 0; damage can break partition interpretation.	Backup structures help recovery and validation; mismatch between primary and backup can show tampering.

7. Part F – File System Identification and Interpretation

From the `lsblk -f` and `df -Th` output in Figure 3, I identified the following file systems on my system:

Volume / mount	File system	Type of storage	Notes
/dev/sda1 mounted on /	ext4	Real disk partition	Root file system, label "root", UUID 4111f04f-67ad-4d5d-a01b-b6cc151feb35, 79G size, 47G used, 28G available (63%)
udev mounted on /dev	devtmpfs	Virtual (RAM)	Device nodes created by the kernel; 865M
tmpfs on /run, /dev/shm, /tmp, /run/user/1000	tmpfs	Virtual (RAM)	Temporary data held in memory; lost at shutdown
none on /run/credentials/…	tmpfs	Virtual (RAM)	Credential directories for systemd services

Only one volume (/dev/sda1) lives on the physical disk, so ext4 is the only disk-based file system on this VM. The others are memory-based and are examples of volatile evidence.

File system	Common OS / usage	Forensic importance
FAT/FAT32	Removable media and older systems	Simple structure; often used on USB drives.
NTFS	Modern Windows systems	Has MFT entries, timestamps, permissions and journals.
ext family (ext4)	Linux systems	Uses inode-based metadata; supports forensic timeline analysis.
XFS/JFS/btrfs	Linux/Unix-like systems	Advanced features may affect recovery and metadata interpretation.
HFS+	Older macOS systems	Important in Mac forensic investigations.

> **Why the file system matters in digital forensics**
> The file system decides how data is organized, named, timestamped and deleted, so an examiner must know it to interpret the disk correctly. On ext4, each file is described by an **inode** that stores size, permissions, owner and the MAC timestamps (modified, accessed, changed), and the journal records recent metadata changes. Knowing this allows an examiner to build timelines, recover deleted files, and find hidden data in slack or unallocated space. Using the wrong assumptions about the file system can lead to wrong findings.

8. Part G – Final Practical Assignment

Partition sizes were verified from the partition table (fdisk and parted output) and calculated step by step in Section 4.2.

Task	Required evidence	Student notes / answer
Identify partition 1	Partition table output (Figures 2 and 5)	/dev/sda1, primary, ext4, Linux (Id 83), boot flag set

Task	Required evidence	Student notes / answer
Record partition 1 sector count	Start, end, sector count	Start 2048, end 167,968,749, sector count 167,966,702
Calculate partition 1 size	Formula and result in GB	167,966,702 × 512 / 1024 / 1024 / 1024 = 80.09 GB (85,998,951,424 bytes)
Identify partition 2	Partition table output	Not present. The disk has only one partition, so none was created (read-only lab).
Record partition 2 sector count	Start, end, sector count	Not applicable. For comparison: whole disk /dev/sda = 167,968,750 sectors; unallocated gap before sda1 = 2,048 sectors.
Calculate partition 2 size	Formula and result in GB	Not applicable. Whole disk: 167,968,750 × 512 / 1024^3 = 80.09 GB; gap: 2,048 × 512 = 1 MiB.
Compare with OS-reported size	fdisk, parted, lsblk, df	fdisk: 80.1G (partition), 80.09 GiB (disk); parted: 86.0GB; df: 79G (see table below).
Explain difference if any	Short explanation	See explanation below: GB vs GiB, and ext4 overhead.

Comparison of calculated and reported sizes

Source	Reported size	Unit system	Comparison with my calculation
My calculation (sda1)	80.09 GB	Binary (GiB)	Reference value
fdisk -l (sda1)	80.1G	Binary (GiB), rounded	Matches (80.09 rounds to 80.1)
fdisk -l (whole disk)	80.09 GiB / 86,000,000,000 bytes	Binary / bytes	Matches Calculation 2 exactly
parted -l	86.0GB	Decimal (GB)	Same bytes: 85,998,951,424 / 10^9 = 86.0 GB
df -Th (/)	79G	Binary (GiB)	Smaller by about 1 GiB: file system overhead

Explanation of the differences

1. GB versus GiB. Manufacturers and parted use decimal GB (1 GB = 1,000,000,000 bytes), while fdisk and my formula use binary GiB (1 GiB = 1,073,741,824 bytes). The same 85,998,951,424 bytes are 86.0 GB in decimal and 80.09 GiB in binary, so the numbers 86.0 and 80.09 do not conflict.

2. File system overhead. df reports 79G for the mounted ext4 file system, about 1 GiB less than the 80.09 GiB partition. ext4 uses part of its partition for its own structures such as the superblock, group descriptors, inode tables and the journal, and this space is not counted as usable size.

3. Reserved blocks. In Figure 3, Used (47G) plus Avail (28G) is 75G, which is less than the 79G size. By default ext4 reserves about 5% of the blocks for the root user, and this explains the remaining gap.

4. Rounding. fdisk shows 80.1G because it rounds 80.09 to one decimal place.

9. Conclusion

In this lab I carried out a safe, read-only examination of a computer system and documented it in the way a forensic trainee should. I identified the platform as a Kali Linux 6.17.10 virtual machine on VMware, mapped five evidence layers, and observed running processes with their PIDs and parent processes. I then calculated the size of the disk and of partition 1 from raw sector counts (167,966,702 sectors × 512 bytes = 85,998,951,424 bytes = 80.09 GB) and checked the result against fdisk, parted and df. The calculation matched the tools once the difference between GB and GiB and the ext4 overhead were taken into account.

I also learned that the disk uses an MBR (msdos) partition table with one ext4 partition, that this points to a BIOS/Legacy boot, and that the first 2,048 sectors before the partition hold the MBR and alignment space. One limitation is that my VM has only one partition, so partition 2 could not be measured, and I did not create one because the lab forbids changes to the disk.

This matters in digital forensics because evidence must be located, interpreted and verified without altering the original. Understanding sectors, partition tables, boot records and file systems allows an examiner to find all data on a disk (including unallocated areas), to prove that sizes and offsets are correct, and to explain findings so that they can be repeated and defended.

10. Screenshot Checklist

No.	Screenshot required	Evidence	Included?
1	System Information or systeminfo output	Figure 1 (uname -a, hostnamectl)	Yes
2	Disk drive model and size output	Figure 2 (fdisk -l)	Yes
3	Logical volume / file system output	Figure 3 (lsblk -f, df -Th)	Yes
4	Running process / PID output	Figure 4 (ps)	Yes
5	fdisk or parted partition table output	Figures 2 and 5 (fdisk -l, parted -l)	Yes
6	Partition 1 calculation evidence	Section 4.2, Calculation 1 (step by step)	Yes
7	Partition 2 calculation evidence	Section 4.2: no second partition exists; whole disk and gap calculated instead	N/A
8	Boot mode evidence: BIOS/UEFI	Figures 2 and 5 (dos / msdos label), Section 6.1	Yes
9	File system identification evidence	Figure 3 (ext4, tmpfs, devtmpfs)	Yes
10	Final report summary / conclusion page	Section 9	Yes